

게임 서버는 어떻게 실시간으로 통신할까?

HTTP Polling부터 WebSocket까지

멀티플레이어
이동 동기화

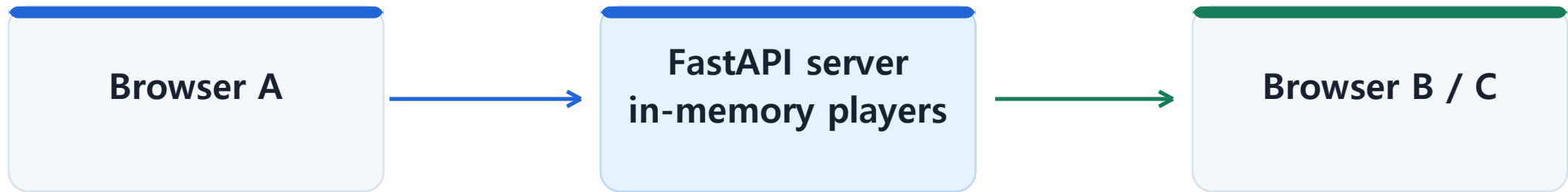
먼저 같이 움직여 봅시다

1. 이름을 입력하고 입장
2. WASD / 방향키로 이동

다른 참가자의 화면에서
내 플레이어가 움직이는가?

질문: Player A의 이동은 Player B에게 어떻게 전달될까?

방금 일어난 일: 공유 상태가 바뀌었다



- MOVE → 서버 상태 갱신 → STATE를 연결된 브라우저에 전송

멀티플레이어에서 생기는 질문

- Player B가 아직 요청하지 않았는데, 서버는 A의 이동을 어떻게 알릴까?
- 실시간성은 변화가 생겼을 때 필요한 지연 안에 전달되는가의 문제다.

HTTP는 보통 클라이언트가 시작한다



- 일반 HTTP: 클라이언트가 요청을 시작하고 서버가 응답한다.
- B가 요청하지 않는 동안 A가 움직이면, B에게 보낼 계기가 없다.

가장 단순한 해결: HTTP Polling

```
setInterval(fetchState, 100);
```

- 클라이언트가 일정 간격으로 GET /api/state를 반복한다.
- 상태가 바뀌지 않아도 요청과 응답은 계속 생긴다.

Polling 간격은 지연과 요청 수의 교환이다

Polling interval	반영 대기	요청 빈도
1000 ms	길어질 수 있음	낮음
500 ms	중간	중간
100 ms	짧아질 수 있음	높음

환경과 사용자 행동에 따라 실제 값은 달라진다.

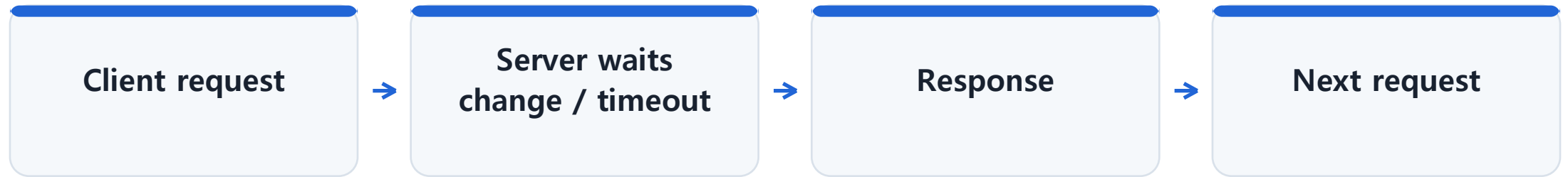
20명이 100 ms마다 물으면?

**20 clients
× 10 polls / second**

**약 200
state requests / second**

상태 요청만의 단순 계산 - 벤치마크 결과가 아님

Long Polling: 응답을 바로 끝내지 않는다



응답 뒤에는 다음 HTTP 요청을 새로 만든다.

SSE: 서버에서 클라이언트로 흐르는 스트림

방식	주된 흐름	이 게임에서의 의미
Polling	Client → Server 반복	주기적 상태 확인
Long Polling	요청 대기 후 응답	빈 요청 감소
SSE	Server → Client	단방향 stream
WebSocket	Client ↔ Server	지속 양방향

WebSocket은 Polling을 감춘 것이 아니다

HTTP Polling
새 HTTP 요청을 반복

WebSocket
하나의 지속 연결에서 frame 교환

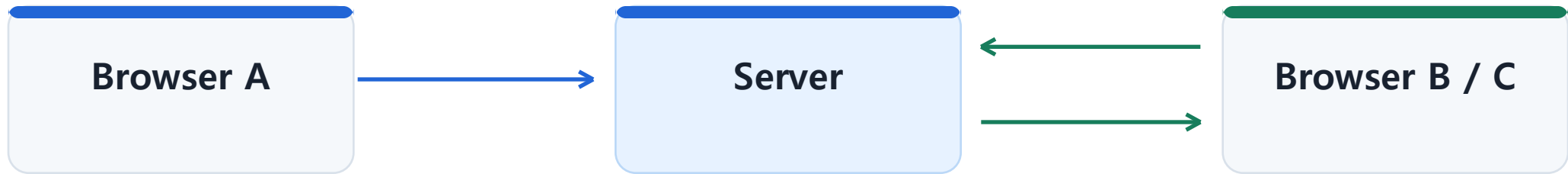
서버는 새 HTTP state request를 기다리지 않고 메시지를 보낼 수 있다.

시작은 HTTP Upgrade handshake다



Upgrade 이후 같은 TCP 연결 위에서 WebSocket protocol의 frame을 주고받는다.

연결을 유지하며 WebSocket frame을 주고받는다



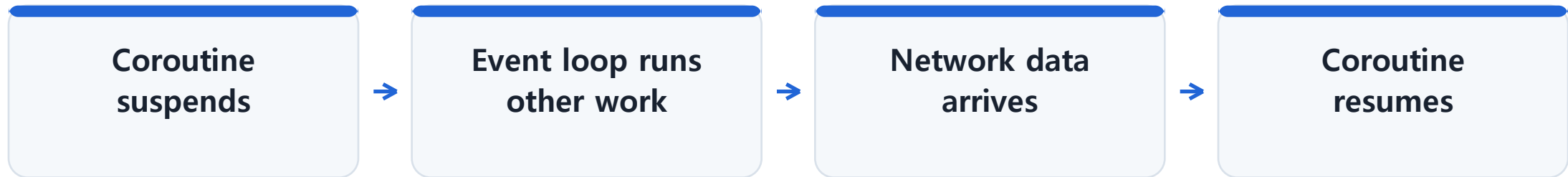
- Client MOVE frame → server
- Server STATE frame → connected clients
- Full-duplex: 양쪽 모두 필요할 때 전송

`while True`면 서버가 계속 확인하는 것 아닌가?

```
while True:  
    data = await websocket.receive_json()
```

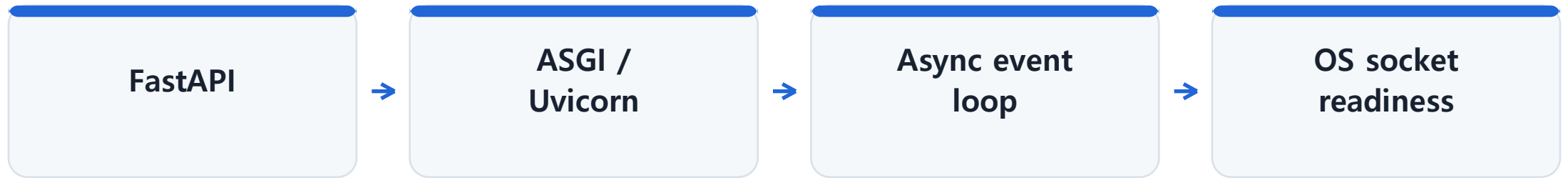
- 이것도 HTTP Polling인가?
- 아니다. 핵심은 `await`가 현재 coroutine을 suspend한다는 점이다.

`await receive\_json()`은 busy polling이 아니다



반복 HTTP request도, CPU가 소켓을 계속 확인하는 busy waiting도 아니다.

FastAPI 아래에서 기다림이 이어지는 층



TCP connection은 OS socket 아래의 전송 계층이다.

epoll은 HTTP Polling과 다른 층의 말이다

HTTP Polling
애플리케이션 수준에서
반복 GET /state

epoll 등 I/O readiness
OS 수준에서 socket 준비 상태를
알리는 mechanism

Linux에서는 runtime 구성에 따라 epoll 같은 I/O multiplexing mechanism이 사용될 수 있다.

실제 WebSocket 서버: 연결 → 상태 변경 → broadcast

```
await websocket.accept()
data = await websocket.receive_json()
... update players ...
await broadcast_state()
```

- /ws와 /websocket/ws는 같은 endpoint로 연결된다.
- broadcast_state()는 active_connections 모두에 STATE JSON을 보낸다.

Hands-on: 실행 환경을 준비합니다

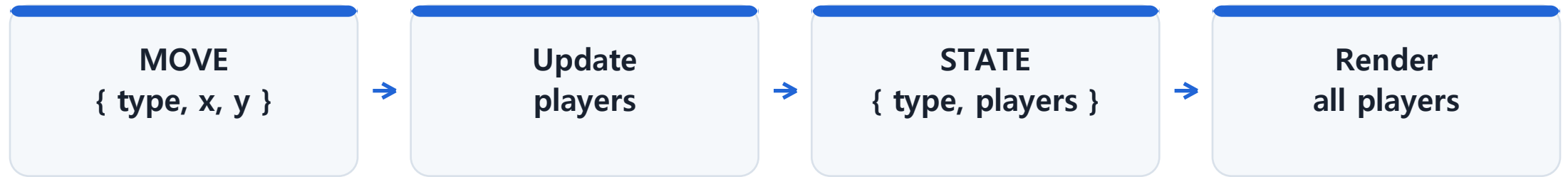
```
cd <workshop-repository>
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app:app --host 0.0.0.0 --port 8000
```

- `http://localhost:8000`을 두 브라우저 창에서 연다.
- 목표: `app.py`의 `broadcast_state()` send loop 완성

Exercise 1: 두 브라우저는 독립 WebSocket client다

- 서버 실행 → 두 창에서 서로 다른 이름으로 join
- 각 브라우저는 별도의 WebSocket connection이다.
- 선택: DevTools Network에서 WS connection 관찰

Exercise 2: MOVE와 STATE의 흐름을 추적한다



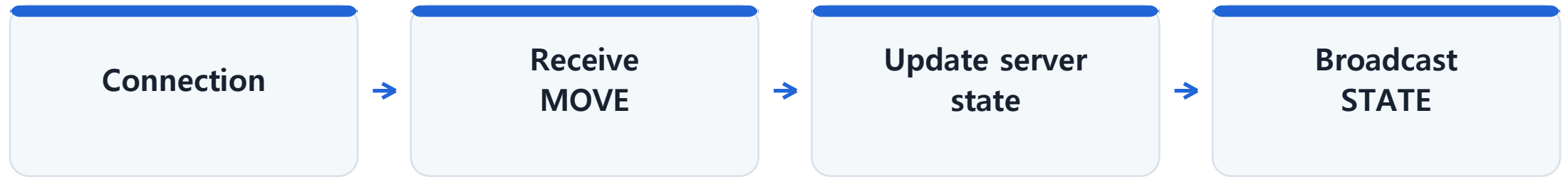
WebSocket move message에는 client-provided player ID가 없다.

Exercise 3: 모든 active connection에 state를 보낸다

```
# TODO: send the updated state
for connection in active_connections:
    await connection.send_json(payload)
```

- 한 창에서 움직이고 다른 창이 갱신되는지 확인한다.
- join, move, disconnect 뒤 모두 broadcast가 필요하다.

Debrief: opening demo도 같은 흐름이었다



WebSocketDisconnect → connection / player 제거 → updated state broadcast

같은 게임을 HTTP Polling으로 바꾸면

HTTP API	역할	실제 서버
POST /api/join	server-generated playerId	join_game
POST /api/move	위치 갱신	move_player
GET /api/state	현재 players 목록	game_state
POST /api/leave	플레이어 제거	leave_game

Public Polling experiment: interval을 바꿔 본다

```
const POLL_INTERVAL_MS = 100;  
pollTimer = window.setInterval(pollState, POLL_INTERVAL_MS);
```

- 1000 ms / 500 ms / 100 ms를 비교한다.
- 긴 interval: 요청 감소, 반영 지연 가능
- 짧은 interval: 더 자주 갱신, 반복 GET 증가

Metrics experiment: 숫자는 오늘의 관찰값으로 채운다

Before each run	WebSocket	Polling
POST /api/metrics/reset	connections / messages	total API / join / move / state / leave
Controlled run	GET /api/metrics	GET /api/metrics

빈칸은 발표 중 실제 관찰값으로 채운다. 사전 benchmark 수치를 제시하지 않는다.

비교의 핵심은 '요청 패턴'이다

Polling
주기적인 GET /api/state
변화가 없어도 반복

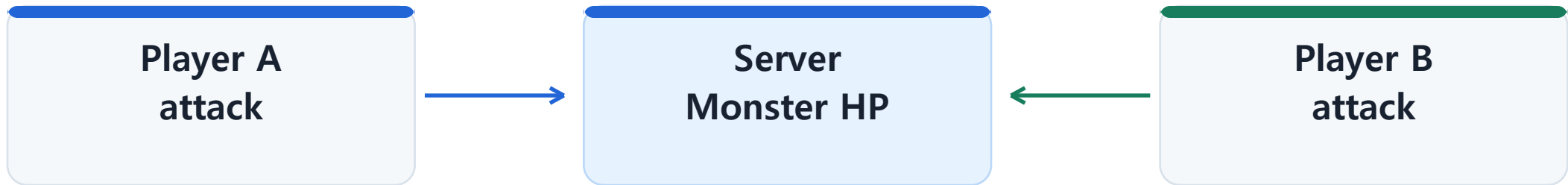
WebSocket
지속 connection
MOVE / STATE message

WebSocket도 broadcast 비용이 0은 아니다. 연결 수만큼 state message가 전송된다.

프로토콜은 문제에 맞춰 고른다

HTTP	WebSocket	선택 기준
Login	Realtime movement	data flow
Shop lookup	Realtime events	latency
Leaderboard	Chat	connection lifetime

WebSocket은 게임 서버의 시작점이다



- Concurrency · tick / game loop · state synchronization · ordering
- Latency compensation · server authority · scaling